

# Assignment 2-Case Study

## Stage 3 Tutorial Activities

### From Requirements to Domain Models

---

Week 6 | 60 minutes

#### Candidate Concepts

| Candidate    | Class?       | Reason                                                                                                                                                       |
|--------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Patient      | Yes          | A real thing in the clinic with its own data (ID, name) and behaviour.                                                                                       |
| Practitioner | Yes          | A real thing in the clinic with its own data and its own appointments.                                                                                       |
| Appointment  | Yes          | Links a patient to a practitioner and holds a date, time and status.                                                                                         |
| Name         | No           | An attribute of Patient and Practitioner, not a thing with its own behaviour.                                                                                |
| Clinic       | Maybe        | There is only one clinic. It is a class only if something must coordinate the others, so it is not needed yet.                                               |
| Database     | No           | A storage technology, not a domain concept.                                                                                                                  |
| Cancellation | No (for now) | An action on an Appointment, handled by a cancel operation and the Cancelled status. It would become a class only if we must store details such as a reason. |
| Status       | No           | An attribute of Appointment (Booked, Cancelled or Completed).                                                                                                |

#### CRC Cards

##### Patient

|                                                |               |
|------------------------------------------------|---------------|
| Responsibilities                               | Collaborators |
| Hold the patient's ID and name                 | (none)        |
| Be found by ID or name and provide its details | Appointment   |

##### Practitioner

|                                                      |               |
|------------------------------------------------------|---------------|
| Responsibilities                                     | Collaborators |
| Hold the practitioner's ID and name                  | (none)        |
| Provide its appointments and availability for a date | Appointment   |

## Appointment

| Responsibilities                                              | Collaborators         |
|---------------------------------------------------------------|-----------------------|
| Hold its patient, practitioner, date and time                 | Patient, Practitioner |
| Know its status and be cancelled while staying in the history | (none)                |

## Relationship Reasoning

Patient to Appointment: Association, one patient to many appointments (1 to 0..\*), and each appointment is for exactly one patient. An appointment refers to a patient but is not part of the patient, and both exist independently, so it is not composition.

Practitioner to Appointment: one practitioner to many appointments (1 to 0..\*), and each appointment has exactly one practitioner.

Should Appointment inherit from Patient? No. An appointment is not a kind of patient, so inheritance is wrong. It has a patient, which is an association.

Does Clinic need to own every object? No. Making Clinic control everything makes it an overloaded "god class" and takes responsibilities away from Patient, Practitioner and Appointment. Clinic is only worth having if something genuinely needs to coordinate them.

## AI Model Critique

PatientManager, PractitionerManager, AppointmentManager: Modify or defer. Separate manager classes that hold all the logic strip behaviour out of the domain classes. Keep the rules in Patient, Practitioner and Appointment for now. A single service or storage class may be reasonable later, but the requirements do not need three managers.

ClinicController: Reject for the domain model. A controller is an application or architecture idea, not a domain concept, and it overlaps with Clinic. There is no requirement for it yet.

NotificationManager: Reject. Notifications and reminders were never requested by the client and were marked out of scope. There is no evidence for it.

ScheduleEngine: Reject as premature. The requirement is to block double-bookings, which is a rule the system can enforce when creating an appointment. Nothing in the brief asks for an automated scheduling engine.